

Road Network Description File Annotation

Road Network Description File Annotation

1. Course Content
2. Road Network Annotation
 - 2.1 Starting the Navigation Node
 - 2.2 Adding a Waypoint
 - 2.3 Adding Edges
3. Save the road network description file
4. Loading and Modifying Road Network Description
 - 4.1 Loading the Road Network Description File
 - 4.2 Editing the Road Network

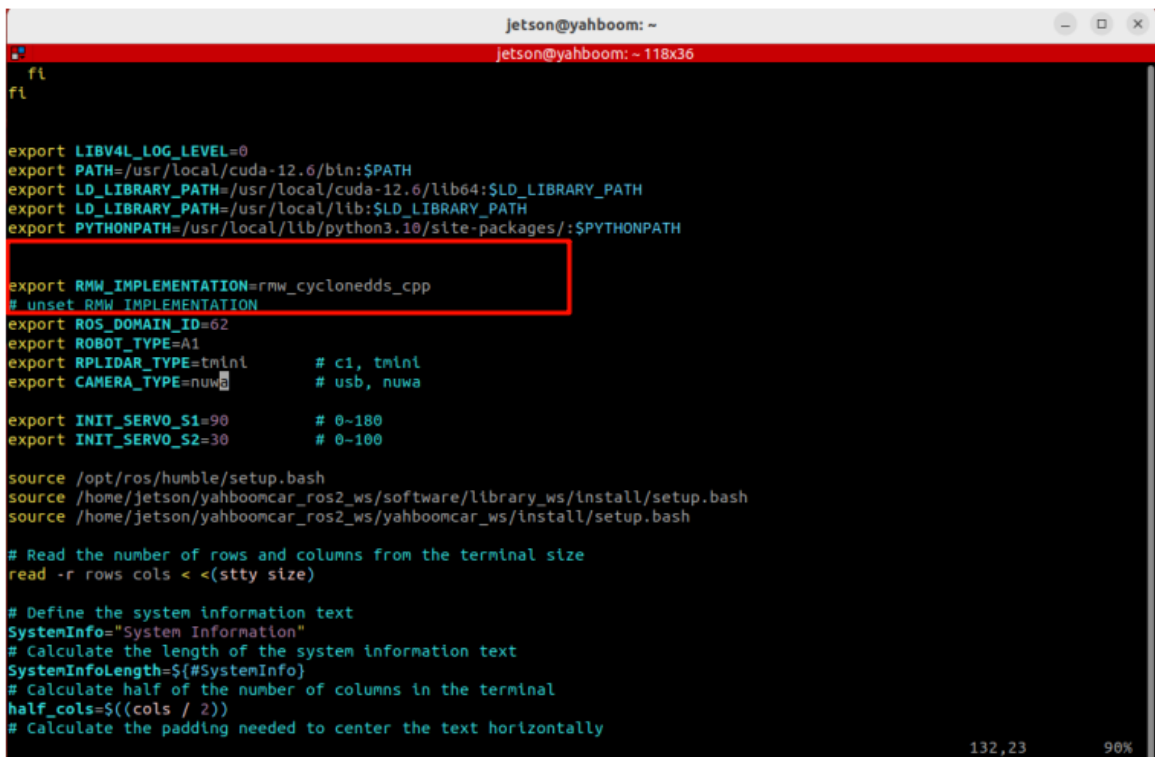
1. Course Content

1. Using the route_tool tool to annotate the .geojson format file describing the road network

2. Road Network Annotation

Open ~/.bashrc, uncomment this line, and change it as shown in the image. (After learning the road network planning function, please restore this comment to avoid affecting the previous examples.)

```
:wq #Save and exit
source ~/.bashrc #Update environment
```



A terminal window titled 'jetson@yahboom: ~' showing the contents of a .bashrc file. The file contains various environment variables for ROS and other tools. A red box highlights the line 'export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp' and the line '# unset RMW_IMPLEMENTATION' below it. The terminal also shows the output of 'source' commands and some terminal size information.

```
jetson@yahboom: ~
jetson@yahboom: ~ 118x36

fl
fl

export LIBV4L_LOG_LEVEL=0
export PATH=/usr/local/cuda-12.6/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda-12.6/lib64:$LD_LIBRARY_PATH
export LD_LIBRARY_PATH=/usr/local/lib:$LD_LIBRARY_PATH
export PYTHONPATH=/usr/local/lib/python3.10/site-packages:$PYTHONPATH

export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
# unset RMW_IMPLEMENTATION
export ROS_DOMAIN_ID=62
export ROBOT_TYPE=A1
export RPLIDAR_TYPE=tm1nl      # c1, tm1nl
export CAMERA_TYPE=numa       # usb, nuwa

export INIT_SERVO_S1=90        # 0~180
export INIT_SERVO_S2=30        # 0~100

source /opt/ros/humble/setup.bash
source /home/jetson/yahboomcar_ros2_ws/software/library_ws/install/setup.bash
source /home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/install/setup.bash

# Read the number of rows and columns from the terminal size
read -r rows cols < <((stty size))

# Define the system information text
SystemInfo="System Information"
# Calculate the length of the system information text
SystemInfoLength=${#SystemInfo}
# Calculate half of the number of columns in the terminal
half_cols=$((cols / 2))
# Calculate the padding needed to center the text horizontally
```

132,23 90%

2.1 Starting the Navigation Node

- When marking the road network, we need to know the exact location of the vehicle's navigation system on the map. Therefore, we need to use the global positioning of the navigation node here.
- Road Network Visualization on Docker

Start Docker (This needs to be run the first time; subsequent starts don't require this step)

```
cd ros2_kilted/  
docker compose up -d
```

```
jetson@yahboom:~/ros2_kilted$ docker compose up -d  
[+] Running 1/1  
✓ Container ros2_kilted-ros2_kilted-1 Started 1.5s  
jetson@yahboom:~/ros2_kilted$
```

To start Docker subsequently, run the following command in the terminal:

```
bringup_ros2kilted
```

To enter the Docker container, run the following command in the terminal:

```
exec_ros2kilted
```

```
jetson@yahboom:~$ bringup_ros2kilted ①  
access control disabled, clients can connect from any host  
[+] Running 1/1  
✓ Container ros2_kilted-ros2_kilted-1 Started 1.4s  
jetson@yahboom:~$ exec_ros2kilted ②  
root@yahboom:/#
```

Modify `do_main_id` to match the host machine:

```
nano .bashrc
```

After making the changes, press Ctrl+X, then enter 'y' and press Enter to save and exit.

```
root@yahboom: ~
root@yahboom: ~ 80x24
GNU nano 7.2 .bashrc
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
source /opt/ros/kilted/setup.bash
source /root/volumes/code_ws/install/setup.bash
export ROS_DOMAIN_ID=62

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
#if [ -f /etc/bash_completion ] && ! shopt -oq posix; then
#    . /etc/bash_completion
#fi

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Enter the following commands in the terminal for visualization:

```
source ~/.bashrc
ros2 launch road_net_route route_tool.launch.py
```

```
root@yahboom: ~
root@yahboom: ~ 80x24
root@yahboom:~# source ~/.bashrc
root@yahboom:~# ros2 launch road_net_route route_tool.launch.py
[INFO] [launch]: All log files can be found below /root/.ros/log/2025-12-22-08-52-26-237610-yahboom-200
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [rviz2-1]: process started with pid [203]
[rviz2-1] [WARN] [1766393546.440453964] [rmw_cyclonedds_cpp]: Failed to parse type hash for topic 'ros_discovery_info' with type 'rmw_dds_common::msg::dds_::ParticipantEntitiesInfo_' from USER_DATA '(null)'.
[rviz2-1] [WARN] [1766393546.441718017] [rmw_cyclonedds_cpp]: Failed to parse type hash for topic 'rt/parameter_events' with type 'rcl_interfaces::msg::dds_::ParameterEvent_' from USER_DATA '(null)'.
[rviz2-1] [WARN] [1766393546.441774339] [rmw_cyclonedds_cpp]: Failed to parse type hash for topic 'ros_discovery_info' with type 'rmw_dds_common::msg::dds_::ParticipantEntitiesInfo_' from USER_DATA '(null)'.
[rviz2-1] [WARN] [1766393546.441835590] [rmw_cyclonedds_cpp]: Failed to parse type hash for topic 'rt/joy' with type 'sensor_msgs::msg::dds_::Joy_' from USER_DATA '(null)'.
[rviz2-1] [WARN] [1766393546.441861127] [rmw_cyclonedds_cpp]: Failed to parse type hash for topic 'ros_discovery_info' with type 'rmw_dds_common::msg::dds_::ParticipantEntitiesInfo_' from USER_DATA '(null)'.
[rviz2-1] [WARN] [1766393546.441871239] [rmw_cyclonedds_cpp]: Failed to parse type hash for topic 'rt/rosout' with type 'rcl_interfaces::msg::dds_::Log_' from USER_DATA '(null)'
```

- (Virtual machine side) Start visualization

```
ros2 launch road_net_route rviz_only.launch.py tools:=true
```

- (Host side) Navigation node

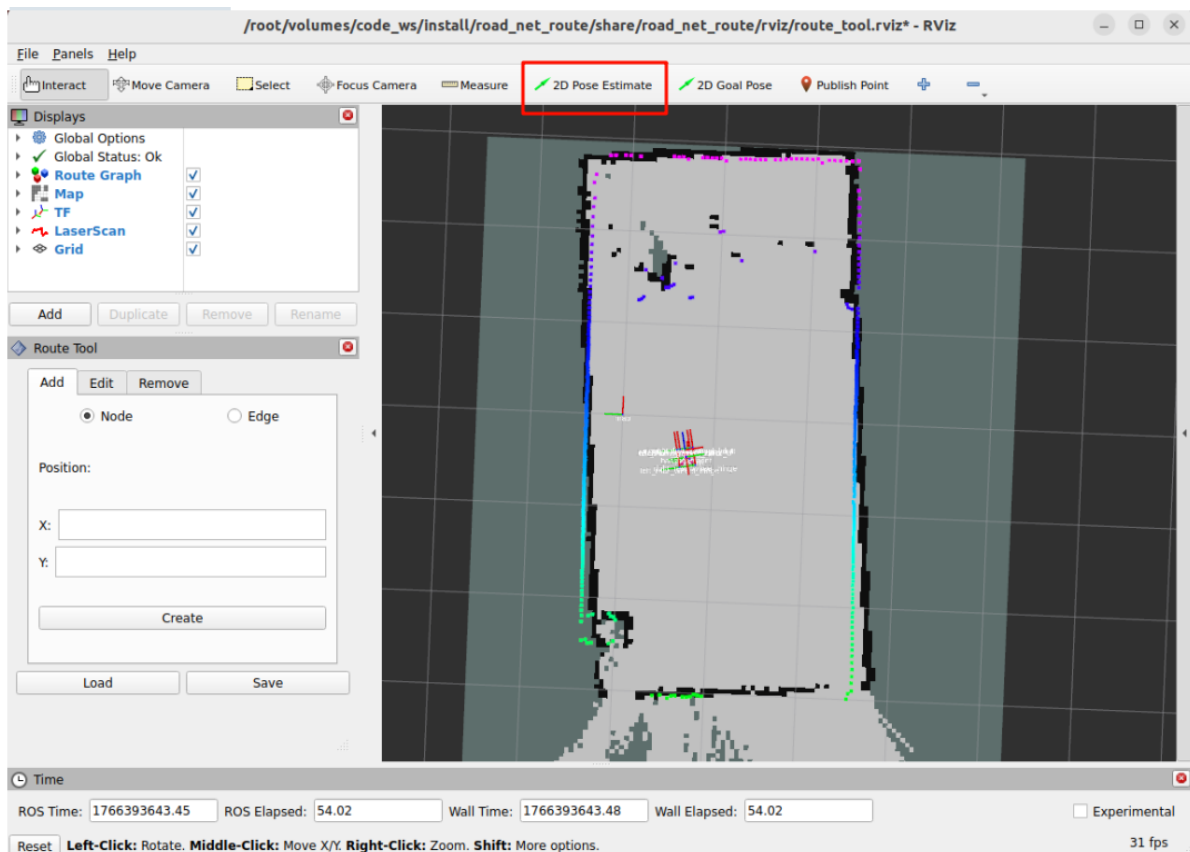
```
#orin
```

```
ros2 launch yahboomcar_nav navigation_teb_launch.py  
map:=/home/jetson/map/map.yaml
```

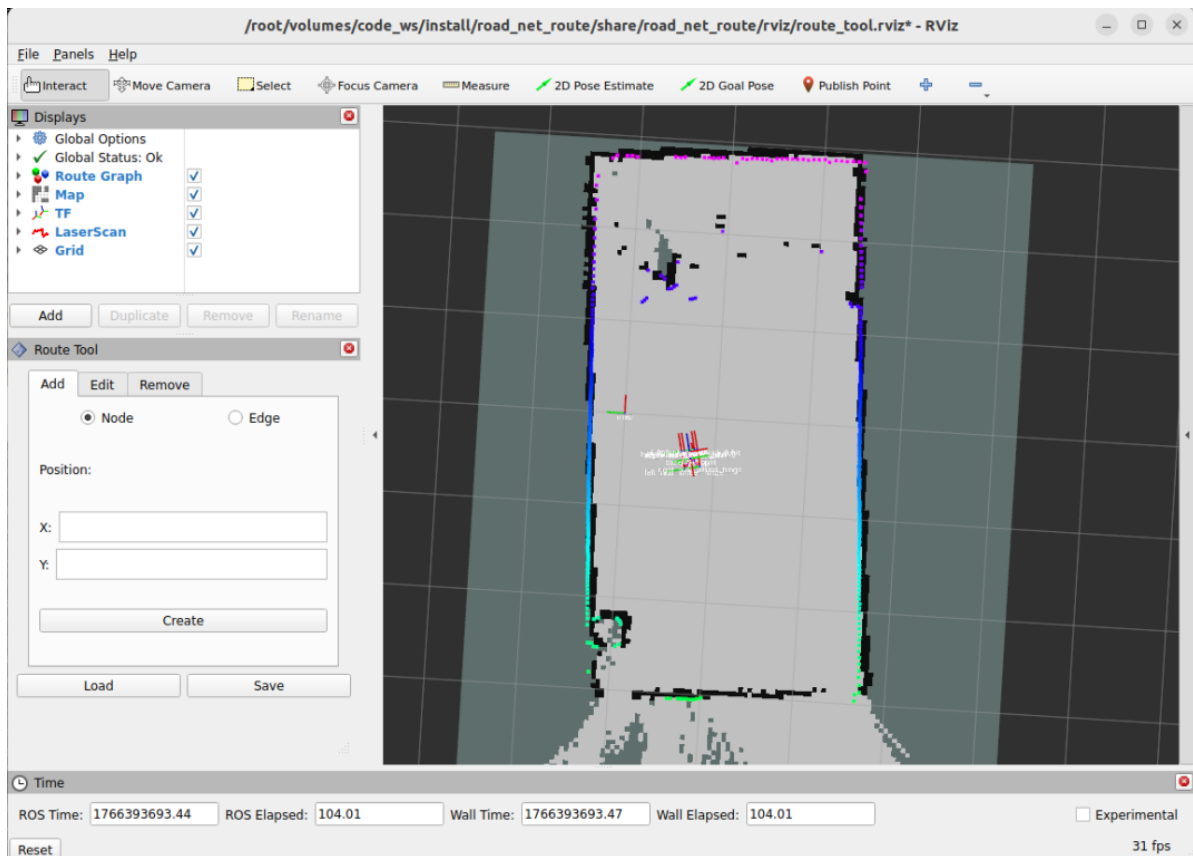
[!NOTE]

Startup parameter description

- **map** The name of the raster map to be loaded; here, we've selected the map saved in the previous lesson.
- The host machine's map is saved by default in the `~/map` path. This path is automatically mounted into the `/root/map` directory within the `ros2_ki1ted` container. Therefore, map files saved on the host machine can be used directly within the `ros2_ki1ted` container.
- RViz will display the map and two coordinate systems. `map` is the map coordinate system, and `base_footprint` is the coordinate system of the projection point of the vehicle's chassis center onto the ground. We first use `2D Pose Estimate` to give the vehicle's initial pose on the map we created, roughly its position on the map.

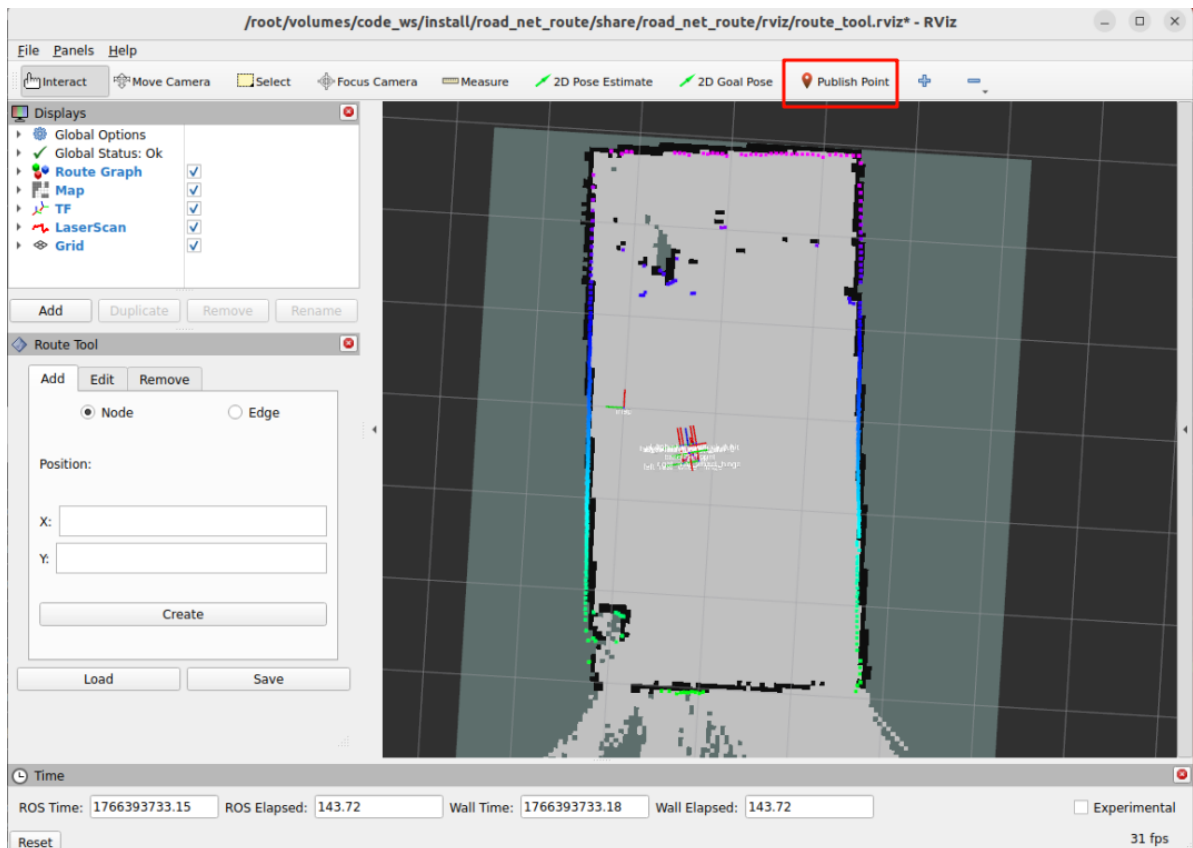


- After giving the approximate position, we can navigate the mobile robot vehicle using keyboard control, gamepad control, or `2D Goal Pose` in RViz, ensuring the radar/laser contour perfectly matches the map.

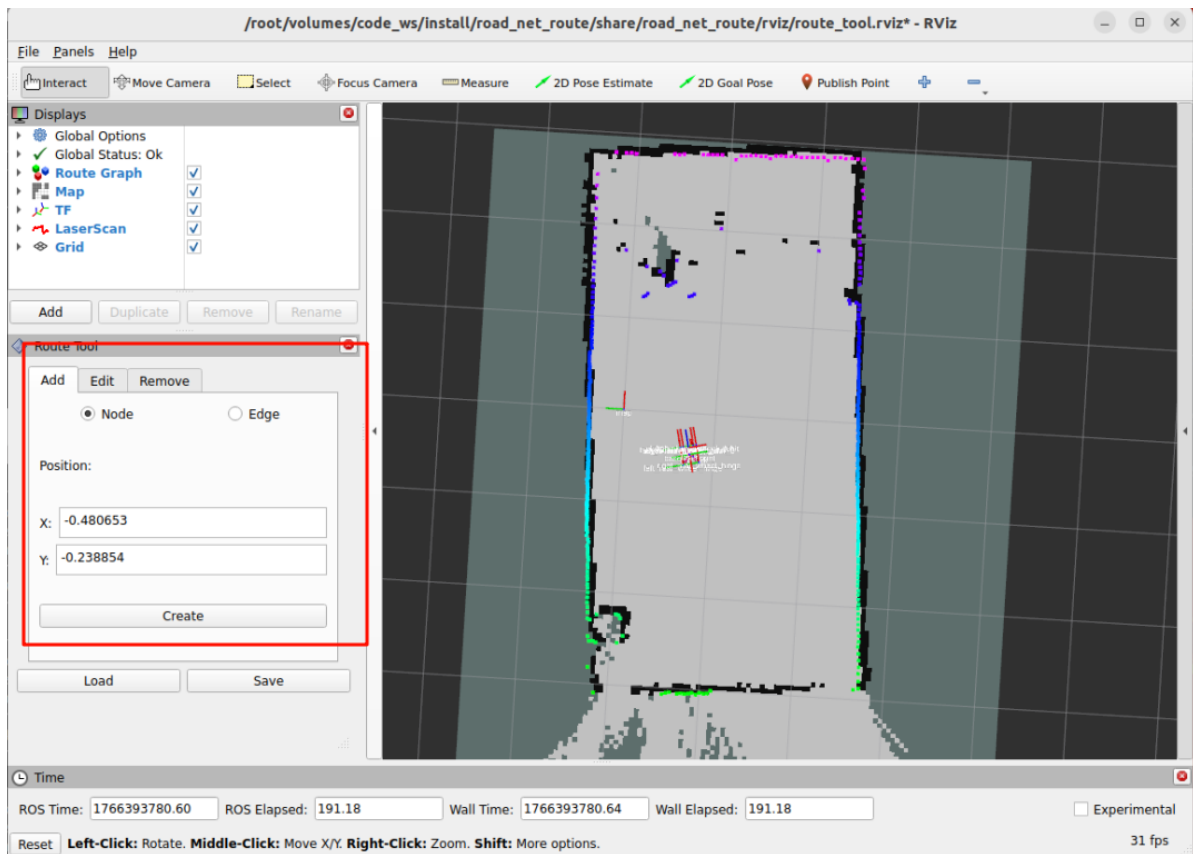


2.2 Adding a Waypoint

- Click the **Publish Point** tool in the RViz toolbar to enter waypoint selection mode.



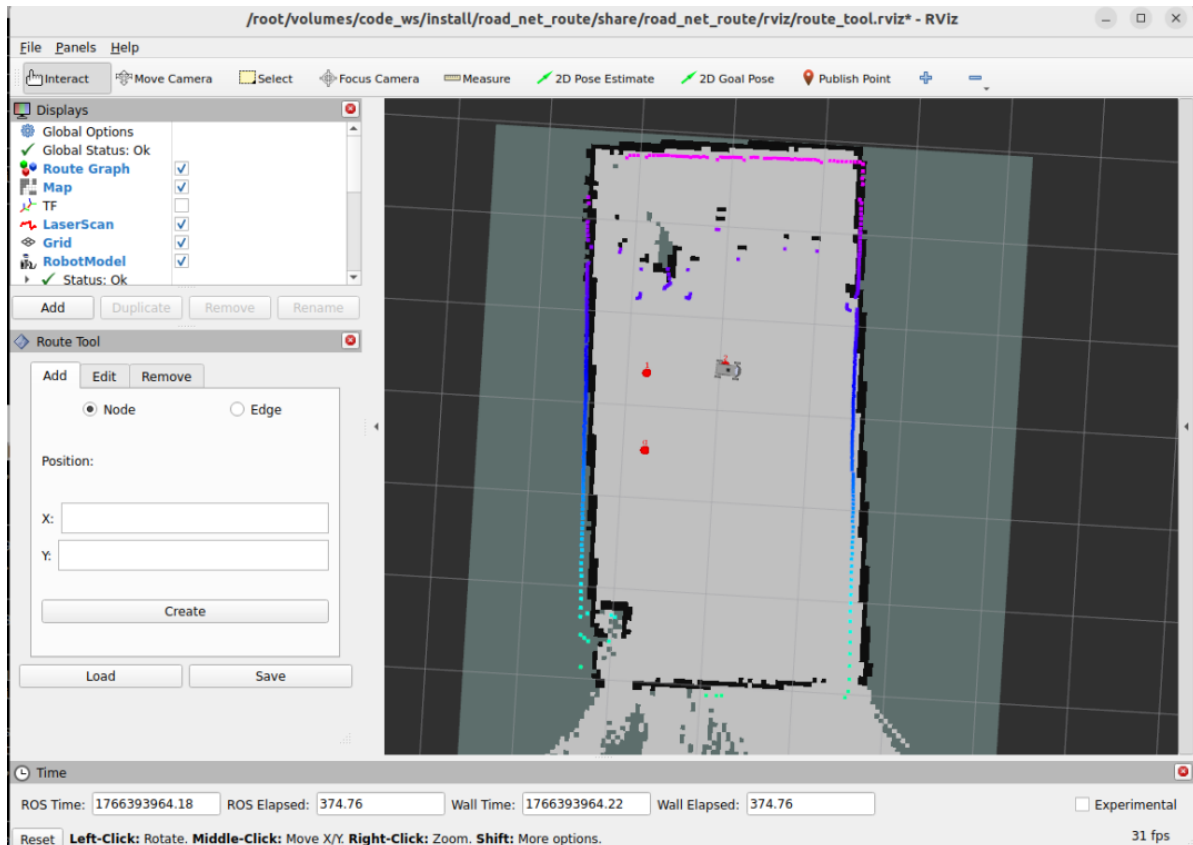
- Then click the left mouse button in the center of the car. The **position** field in the **Route Tool** tab at the lower left will automatically fill in the coordinates of the current point. Then click **Create** to create a waypoint.



- Next, we move the vehicle-mounted robot to the next location where waypoints need to be marked. In this demonstration, we directly move the robot to the next location where waypoints need to be marked.

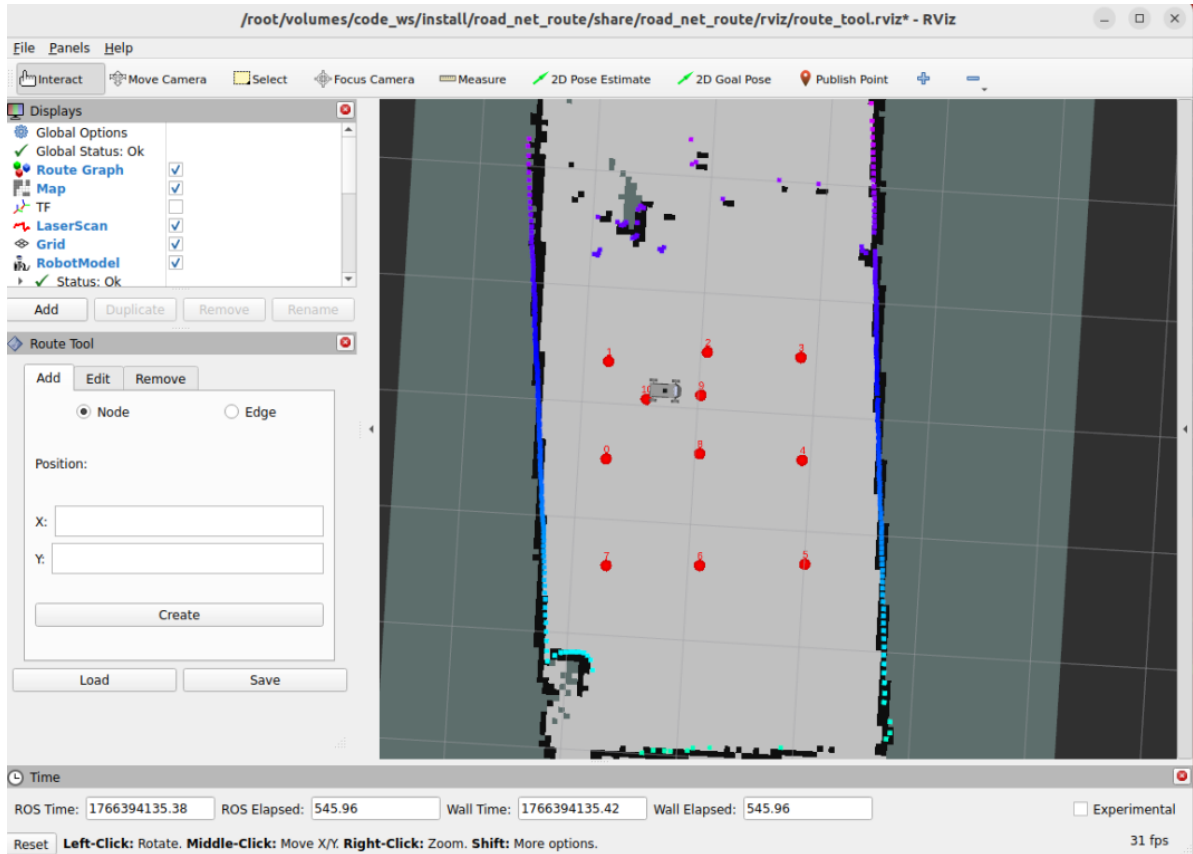
[!TIP]

The mobile robot-mounted robot can be controlled via keyboard, gamepad, or RViz's **2D Goal Pose** navigation.



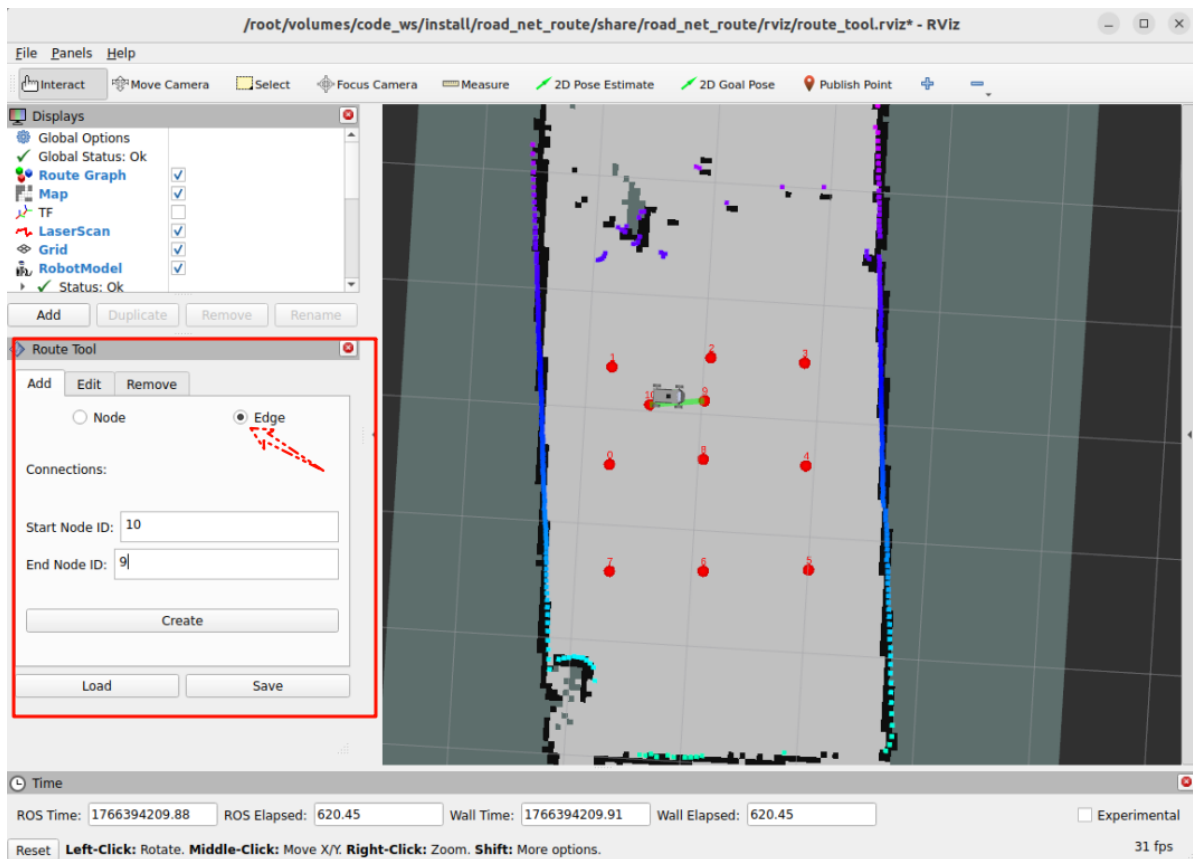
[!WARNING]

- The localization principle here uses AMCL global localization and pose map matching.
- **Note:** If, during subsequent waypoint marking, a discrepancy is observed between the laser contour and map environment features (due to cumulative errors in the odometer or wheel slippage causing local positioning errors), you can first move and rotate the robot a short distance to allow AMCL localization to take effect and ensure the radar laser contour data perfectly matches the map.
- Repeat the above steps until all waypoints need to be marked (mark waypoints according to your actual needs).



2.3 Adding Edges

- Select Edge in the `Route Tool` tab.



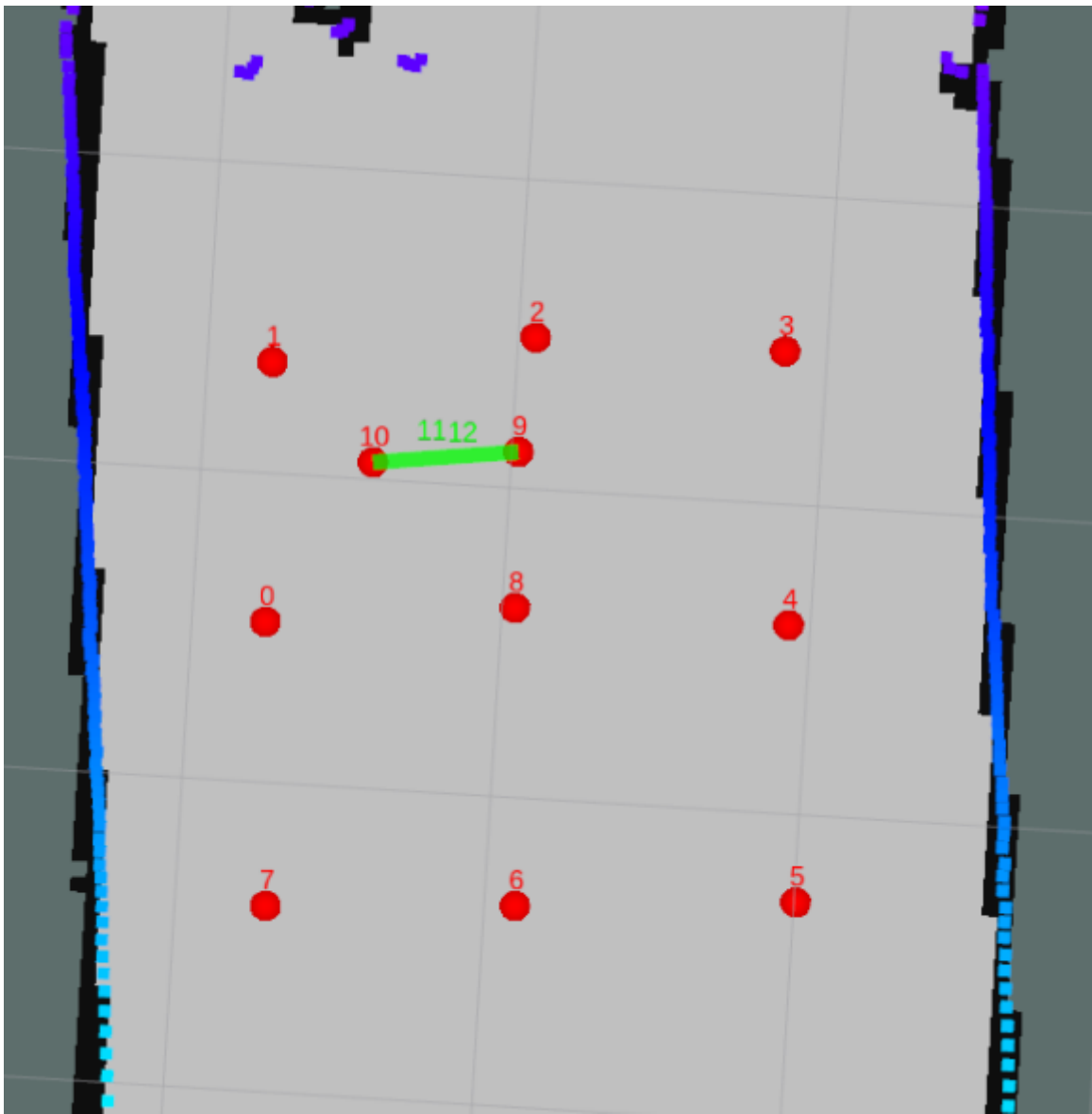
- Enter the starting waypoint in **Start Node ID** and the ending waypoint in **End Node ID**, then click **Create** to create an edge between waypoints.

[!IMPORTANT]

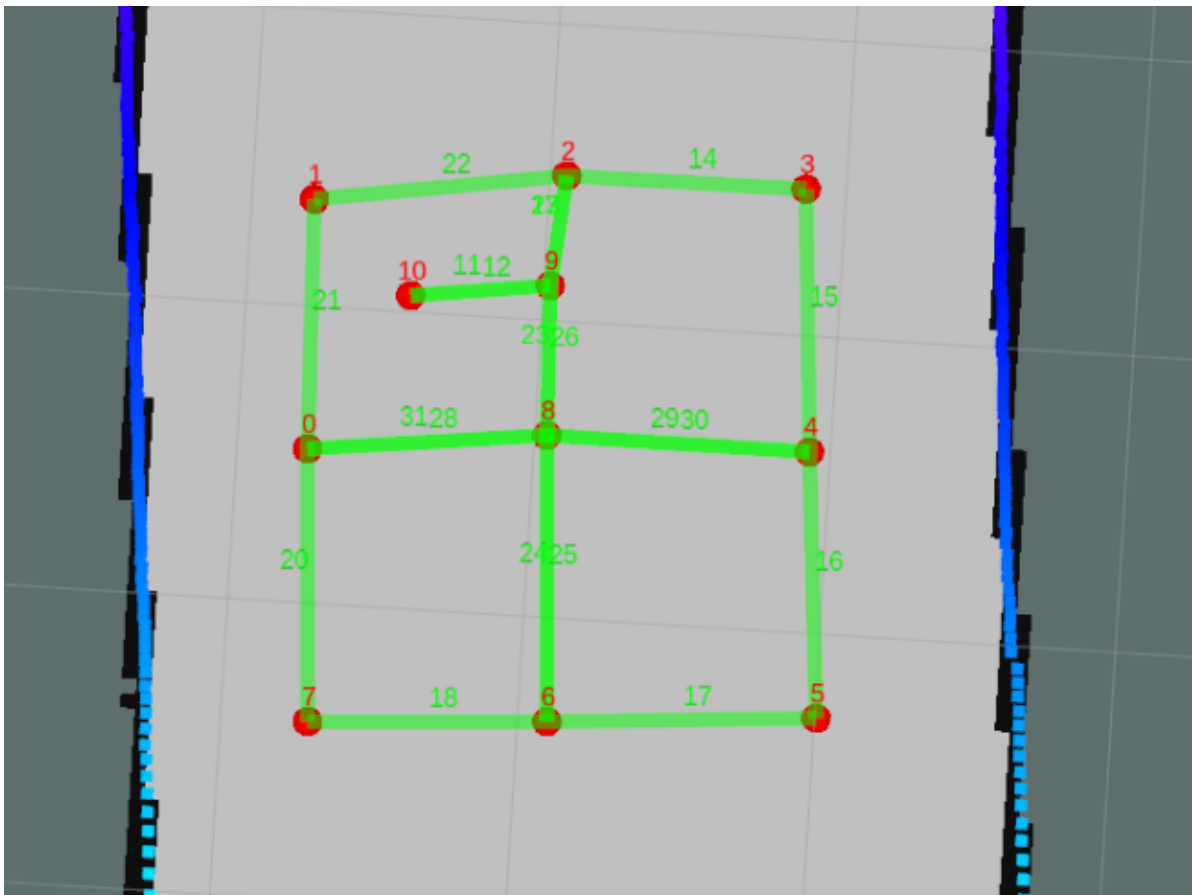
Note!!

A road network is a topological structure composed of nodes and edges, and the edges are directed. Here, the edge from 10-9 is unidirectional; if a bidirectional edge is needed, an edge from 9-10 must also be created.

- Then we will create an edge between points 9-10, allowing for two-way travel between waypoints 9 and 10.

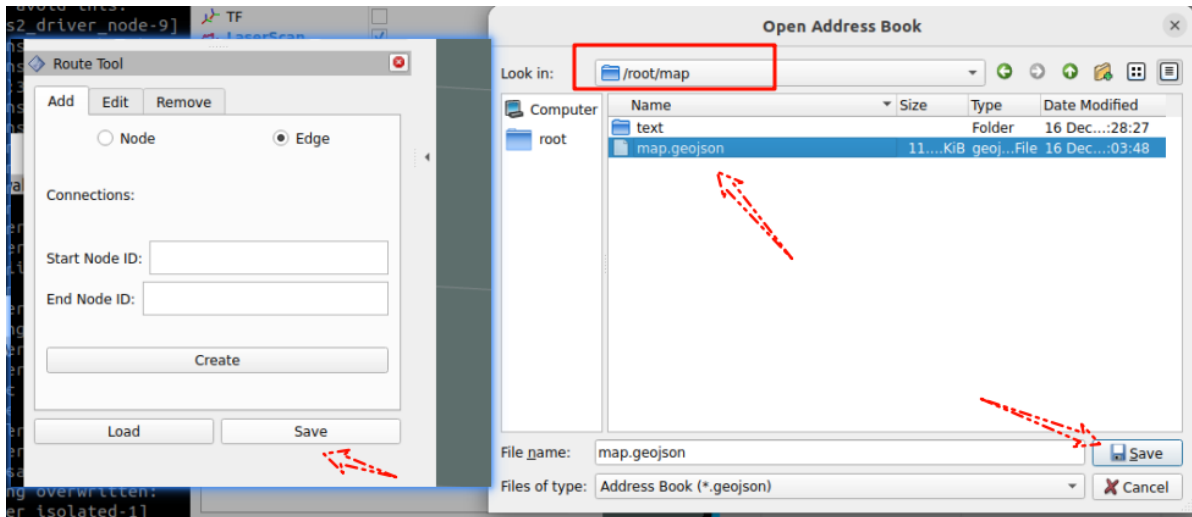


- Repeat the above steps to complete the creation of all edges (whether to create unidirectional or bidirectional edges depends on the user's actual needs).

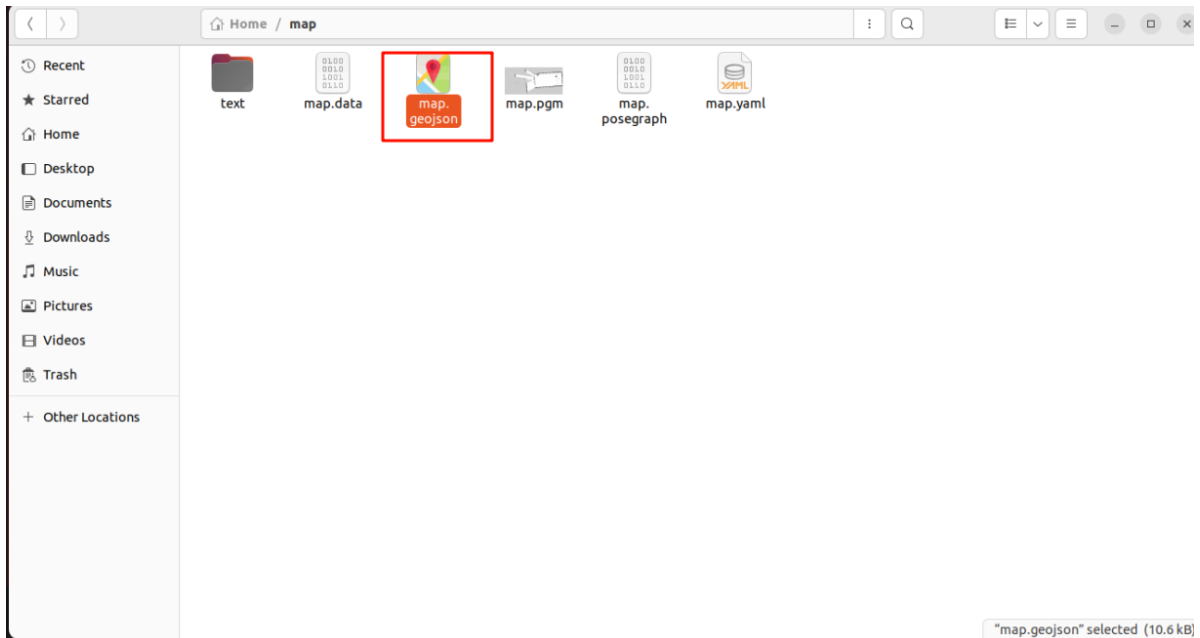


3. Save the road network description file

- In **Route In the Tool** tab, click **Save**. In the pop-up window, select the path to save the file. Here, we choose to directly overwrite `map.geojson` and save it in this path.



- If you don't choose to overwrite, you can also... Enter the file name in the name field. It is recommended to keep it consistent with the raster map name for easy differentiation. The file extension must be `.geojson`. Finally, click **save** to save the road network description file.



- After successful saving, the terminal will print a prompt message.

```
root@yahboom: ~
root@yahboom: ~ 80x24
2
[rviz2-1] [INFO] [1766394396.898381052] [route_tool_node]: Adding edge from 9 to 8
[rviz2-1] [INFO] [1766394401.767536988] [route_tool_node]: Adding edge from 8 to 6
[rviz2-1] [INFO] [1766394405.411045814] [route_tool_node]: Adding edge from 6 to 8
[rviz2-1] [INFO] [1766394408.988780940] [route_tool_node]: Adding edge from 8 to 9
[rviz2-1] [INFO] [1766394413.237411579] [route_tool_node]: Adding edge from 9 to 2
[rviz2-1] [INFO] [1766394418.758806657] [route_tool_node]: Adding edge from 0 to 8
[rviz2-1] [INFO] [1766394422.766498340] [route_tool_node]: Adding edge from 8 to 4
[rviz2-1] [INFO] [1766394427.854977807] [route_tool_node]: Adding edge from 4 to 8
[rviz2-1] [INFO] [1766394431.726255945] [route_tool_node]: Adding edge from 8 to 0
[rviz2-1] [INFO] [1766394500.181958323] [route_tool_node]: Save graph to: /root/map/map.geojson
[rviz2-1] [INFO] [1766394500.182134522] [route_tool_node]: Saving graph file from /root/map/map.geojson, by parser nav2_route::GeoJsonGraphFileSaver
```

- Afterwards, we can view the `map.geojson` road network description file, which contains a JSON format description of waypoints and edge information.

```
1 {
2   "crs": {
3     "properties": {
4       "name": "urn:ogc:def:crs:EPSG::3857"
5     },
6     "type": "name"
7   },
8   "features": [
9     {
10      "geometry": {
11        "coordinates": [
12          -0.48065298795700073,
13          -0.23885400593280792
14        ],
15        "type": "Point"
16      },
17      "properties": {
18        "frame": "map",
19        "id": 0
20      },
21      "type": "Feature"
22    },
23    {
24      "geometry": {
25        "coordinates": [
26          0.3601909875869751,
27          -0.21013300120830536
28        ],
29        "type": "Point"
30      },
31      "properties": {
32        "frame": "map",
33        "id": 1
34      },
35      "type": "Feature"
36    }
37  ]
38 }
```

4. Loading and Modifying Road Network Description

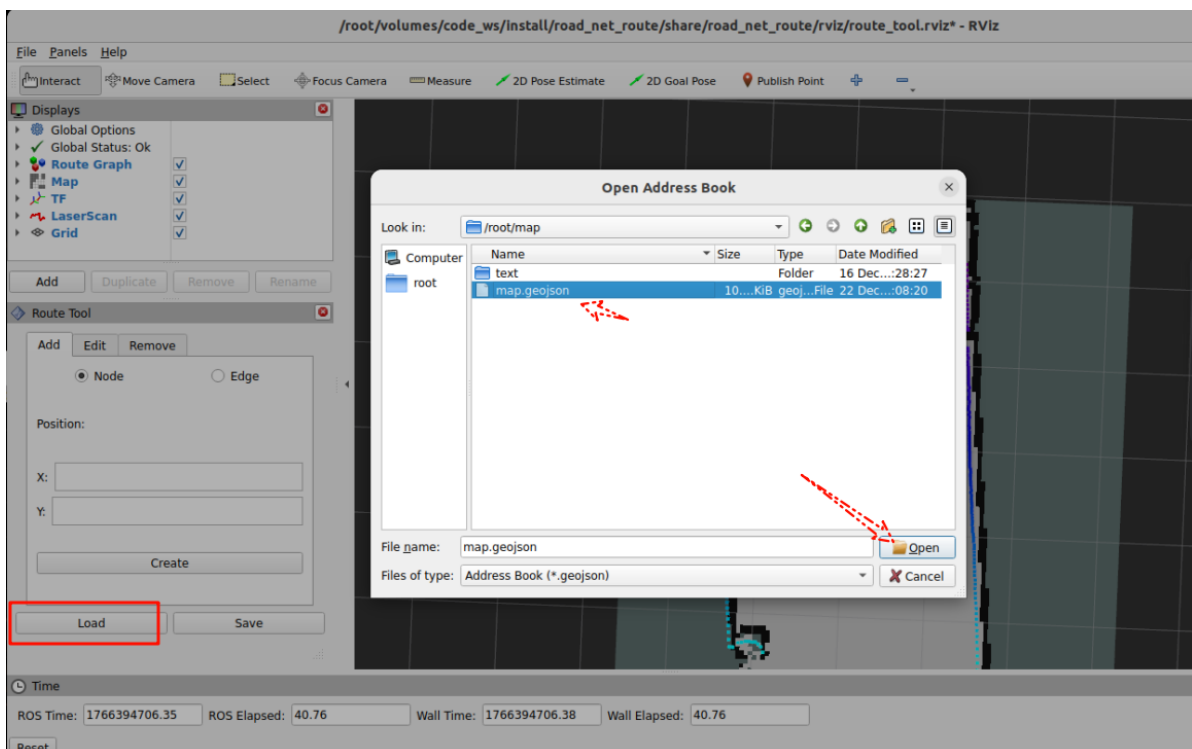
4.1 Loading the Road Network Description File

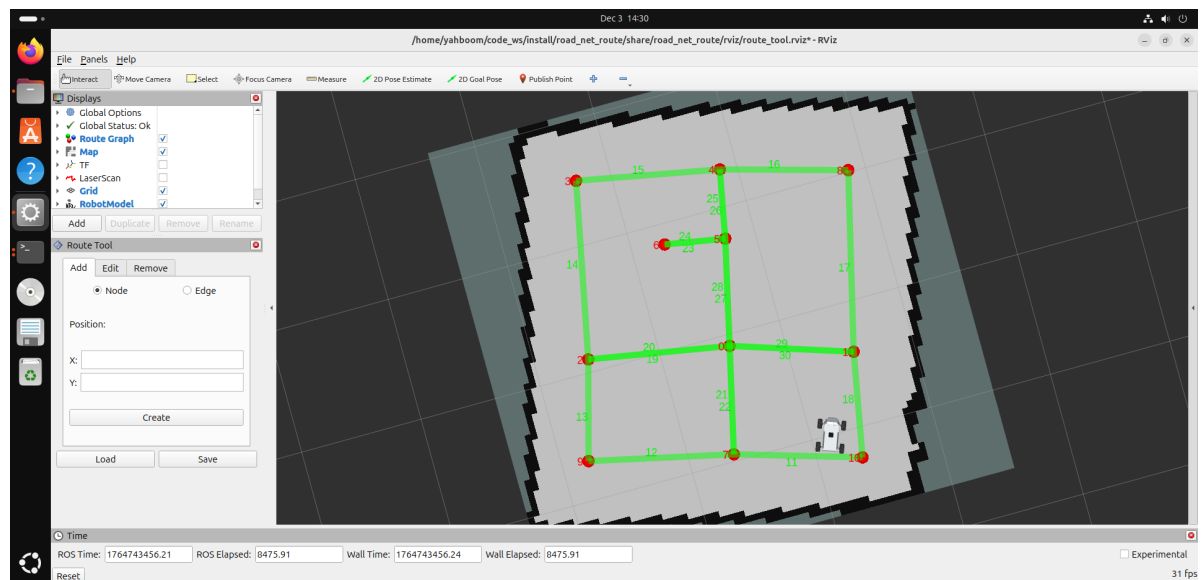
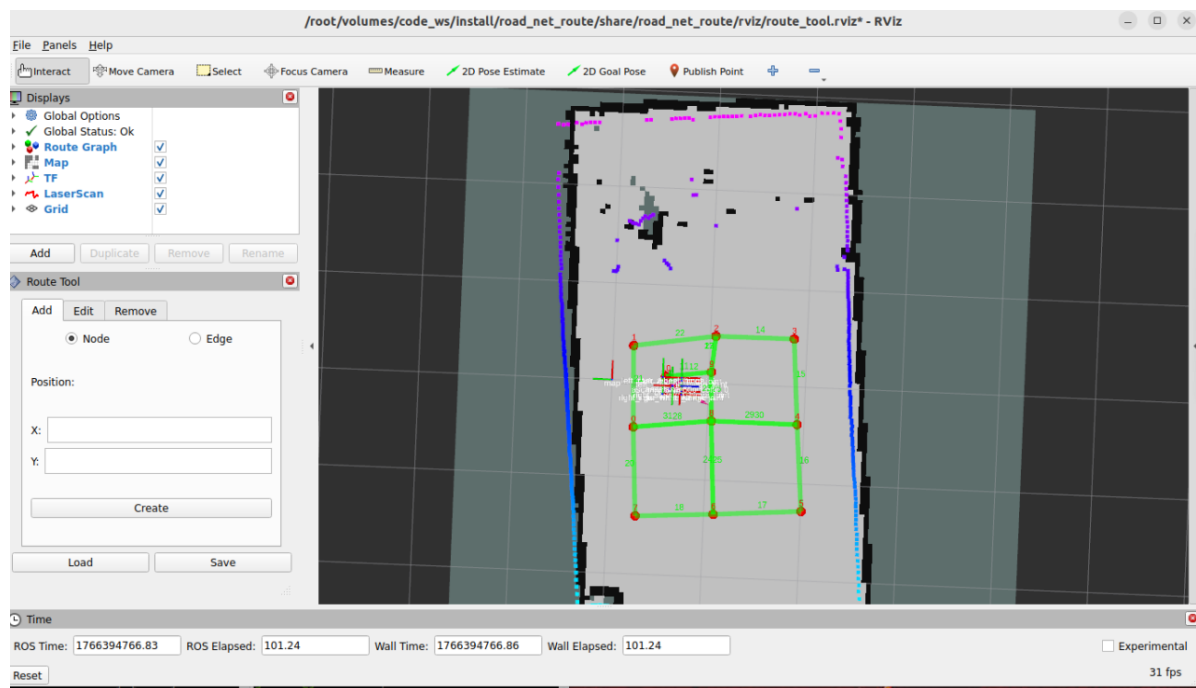
[!TIP]

- If you need to modify the previous road network description file later, such as adding, removing, or editing points and edges, you can load the previous road network description file for editing.

Still following step 2.1 after starting the navigation node,

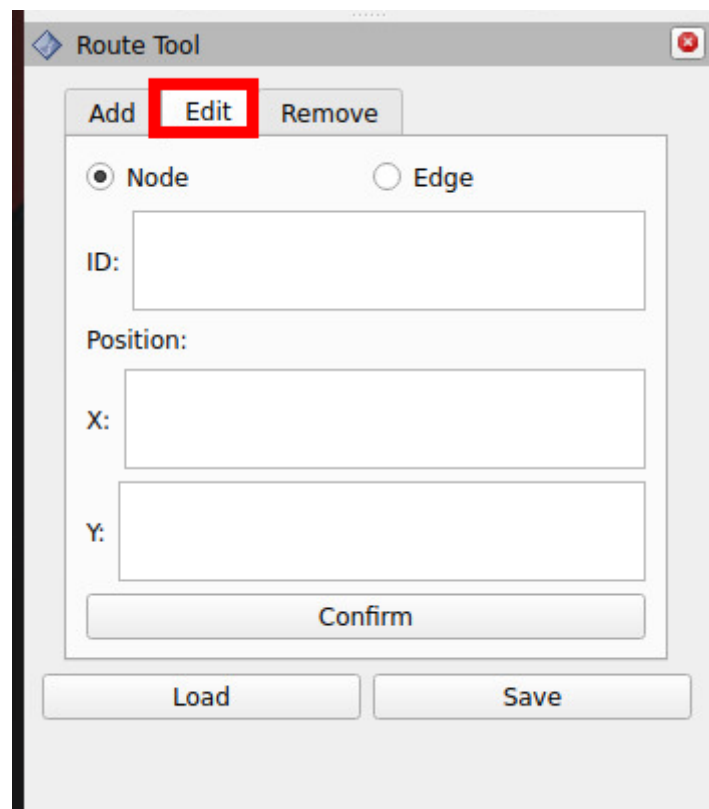
- ✓ In the **Route Tool** tab, click **Load**, then select the road network description file to be edited, and click **open** to load it.





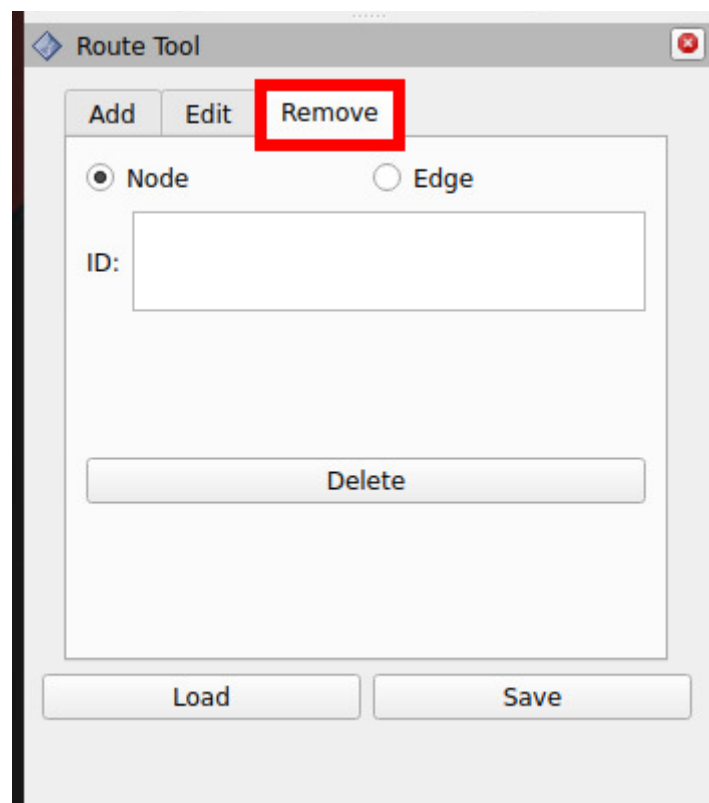
4.2 Editing the Road Network

- If you need to add new waypoints and edges, the operation is the same as before.
- Click **Edit** to edit existing waypoints or edges.



The image shows a 'Route Tool' dialog box. At the top, there are three buttons: 'Add', 'Edit', and 'Remove'. The 'Edit' button is highlighted with a red rectangle. Below these buttons, there are two radio buttons: 'Node' (selected) and 'Edge'. Under 'Node', there is an 'ID:' label followed by a text input field. Below that, there is a 'Position:' label, followed by 'X:' and 'Y:' labels, each with a corresponding text input field. At the bottom of the main content area is a 'Confirm' button. At the very bottom of the dialog are 'Load' and 'Save' buttons.

- Click **Remove** to edit existing waypoints or edges.



The image shows the 'Route Tool' dialog box with the 'Remove' button highlighted by a red rectangle. The 'Add' and 'Edit' buttons are now disabled. The 'Node' radio button remains selected. The 'ID:' text input field is present. The 'Delete' button, which was previously 'Confirm', is now visible. The 'Load' and 'Save' buttons remain at the bottom.